

ใบงานที่ 5 การสร้างโมเดลการเรียนรู้แบบไม่มีผู้สอนด้วย K-Means (Unsupervised Learning)

วัตถุประสงค์การเรียนรู้

1. บอกความหมายและความจำเป็นของการทำ Feature Scaling ในงานด้าน AI ได้
2. เขียนคำสั่ง Python เพื่อปรับสเกลข้อมูล (StandardScaler) ได้ถูกต้อง
3. บอกค่าความคลาดเคลื่อน (Inertia) เพื่อใช้ประกอบการตัดสินใจเลือกจำนวนกลุ่มได้
4. บอกจำนวนกลุ่ม (K) ที่เหมาะสมที่สุดโดยพิจารณาจากกราฟ Elbow Method ได้ถูกต้อง
5. เขียนคำสั่ง Python เพื่อสร้างและสอนโมเดล (Train Model) ด้วยอัลกอริทึม K-Means ได้
6. ระบุชื่อกลุ่มลูกค้า (Cluster Naming) จากค่าเฉลี่ยของจุดศูนย์กลางตามเงื่อนไขที่กำหนดได้
7. เขียนคำสั่ง Python เพื่อบันทึก (Save) และโหลด (Load) โมเดลสำหรับการใช้งานจริงได้
8. เขียนคำสั่ง Python พยากรณ์กลุ่มลูกค้าจากข้อมูลชุดใหม่โดยใช้โมเดลที่บันทึกไว้ได้ถูกต้อง
9. บอกกลยุทธ์การตลาดให้สอดคล้องกับผลการพยากรณ์กลุ่มลูกค้าได้

เครื่องมือและอุปกรณ์

1. เครื่องคอมพิวเตอร์ที่ติดตั้งโปรแกรม PyCharm IDE
2. ไฟล์ข้อมูลดิบ lab3.csv
3. เอกสารใบความรู้หน่วยที่ 5
4. แบบฟอร์มบันทึกผลการปฏิบัติงาน (ท้ายใบงาน)

ลำดับขั้นตอนการปฏิบัติงาน

คำชี้แจง: ให้ผู้เรียนเขียนโค้ดลงในไฟล์ตามขั้นตอนต่อไปนี้

ขั้นตอนที่ 1: การเตรียมและปรับสเกลข้อมูล (Data Preparation)

```
# 1. นำไฟล์ lab3.csv จากโฟลเดอร์ Data management มาเป็น Dataset สำหรับ AI แบ่งกลุ่มลูกค้า
# 2. สร้างไฟล์ใหม่ชื่อ prepare_data.py

import pandas as pd
from sklearn.preprocessing import StandardScaler
import joblib

# 3. โหลดข้อมูลจากไฟล์ lab3.csv
print("กำลังโหลดข้อมูล...")
df = pd.read_csv('lab3.csv')
data = df[['Recency', 'Frequency', 'Monetary']].dropna()

# 4. สร้างตัวแปร Scaler และทำการปรับสเกล (Scaling)
print("กำลังปรับสเกลข้อมูล...")
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

# 5. แปลงผลลัพธ์กลับเป็น DataFrame
df_scaled = pd.DataFrame(data_scaled, columns=['Recency', 'Frequency', 'Monetary'])

# 6. บันทึกไฟล์ข้อมูล (csv) และ โมเดลการสเกล (pkl)
df_scaled.to_csv('customer_scaled.csv', index=False)
joblib.dump(scaler, 'customer_scaler.pkl')

print("บันทึกไฟล์สำเร็จ: customer_scaled.csv และ customer_scaler.pkl")
```

ขั้นตอนที่ 2: การหาค่า K ที่เหมาะสม (Elbow Method)

```
# 1. สร้างไฟล์ใหม่ชื่อ find_k.py
import pandas as pd
from sklearn.cluster import KMeans

# 2. โหลดข้อมูลที่สเกลแล้ว
df = pd.read_csv('customer_scaled.csv')
X = df.values

# 3. วาดรูปหาค่า Inertia ตั้งแต่ K=1 ถึง 10
print("--- ผลการทดสอบ Elbow Method ---")
for k in range(1, 11):
    # กำหนด n_init=10 เพื่อความแม่นยำ
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X)
    # แสดงค่า Inertia ของแต่ละ K
    print(f"K = {k}, Inertia = {model.inertia_:.2f}")
```

ขั้นตอนที่ 3: การสร้างโมเดลและระบุชื่อกลุ่ม (Model Training & Profiling)

```
# 1. สร้างไฟล์ใหม่ชื่อ train_model.py
import pandas as pd
from sklearn.cluster import KMeans
import joblib
import pickle

# 2. โหลดข้อมูล
df = pd.read_csv('customer_scaled.csv')
X = df.values

# 3. สร้างโมเดล K-Means (กำหนด K=3 ตามผลจาก Elbow Method)
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
```

```

df['Cluster'] = kmeans.fit_predict(X)

# 4. วาดรูปแสดงค่าเฉลี่ยของแต่ละกลุ่ม (Centroid Analysis)
print("\n=== วิเคราะห์พฤติกรรมกลุ่ม ===")
for i in range(optimal_k):
    cluster_data = df[df['Cluster'] == i]
    print(f"กลุ่ม {i}: R={cluster_data['Recency'].mean():.2f}, "
          f"F={cluster_data['Frequency'].mean():.2f}, "
          f"M={cluster_data['Monetary'].mean():.2f}")

# 5. ระบุชื่อกลุ่ม (Naming) ตามเงื่อนไข
cluster_names = {}
for i in range(optimal_k):
    cluster_data = df[df['Cluster'] == i]
    avg_r = cluster_data['Recency'].mean()
    avg_f = cluster_data['Frequency'].mean()
    avg_m = cluster_data['Monetary'].mean()

    # Logic การตั้งชื่อ: VIP, Churned, หรือ General
    if avg_m > 0.8 and avg_f > 0.7:
        cluster_names[i] = "💎 VIP (ซื้อเยอะ-บ่อย)"
    elif avg_r > 1.0:
        cluster_names[i] = "⚠️ Churned (หายไปนาน)"
    else:
        cluster_names[i] = "😐 General (ทั่วไป)"

# 6. บันทึกโมเดลและชื่อกลุ่ม
joblib.dump(kmeans, 'customer_kmeans.pkl')
with open("cluster_names.pkl", "wb") as f:
    pickle.dump(cluster_names, f)

print("บันทึกโมเดลเรียบร้อยแล้ว")

```

ขั้นตอนที่ 4: การพยากรณ์และการกำหนดกลยุทธ์ (Prediction & Strategy)

```
# 1. สร้างไฟล์ใหม่ชื่อ run_model.py
import pandas as pd
import joblib
import pickle

# 2. โหลดโมเดล, Scaler และชื่อกลุ่ม
kmeans = joblib.load('customer_kmeans.pkl')
scaler = joblib.load('customer_scaler.pkl')
with open("cluster_names.pkl", "rb") as f:
    names = pickle.load(f)

# 3. สร้างฟังก์ชันพยากรณ์ (Predict Function)
def predict_user(recency, frequency, monetary):
    # เตรียมข้อมูลและ *Scale ด้วย Scaler ตัวเดิม*
    user_data = pd.DataFrame([[recency, frequency, monetary]],
                              columns=['Recency', 'Frequency', 'Monetary'])
    user_scaled = scaler.transform(user_data)

    # พยากรณ์กลุ่ม
    cid = kmeans.predict(user_scaled)[0]
    return names.get(cid, "Unknown")

# 4. กำหนดกลยุทธ์ (Strategy Logic)
def get_strategy(group_name):
    if "VIP" in group_name:
        return "เสนอสินค้า Premium + ส่งฟรี"
    elif "Churned" in group_name:
        return "ส่งคูปองส่วนลด 50% เพื่อดึงกลับมา"
    else:
        return "เสนอโปรโมชั่น ชื้อ 2 แถม 1"
```

5. ทดสอบพยากรณ์

customer_a = [1, 25, 15000] # ลูกค้า A

group_a = predict_user(*customer_a)

print(f"ลูกค้า A: อยู่กลุ่ม {group_a} -> กลยุทธ์: {get_strategy(group_a)}")

customer_b = [365, 1, 100] # ลูกค้า B

group_b = predict_user(*customer_b)

print(f"ลูกค้า B: อยู่กลุ่ม {group_b} -> กลยุทธ์: {get_strategy(group_b)}")

แบบบันทึกผลการปฏิบัติงาน

ส่วนที่ 1: ผลการเตรียมและปรับสเกลข้อมูล

- จำนวนข้อมูล (Rows):
 - ข้อมูลที่โหลดเข้ามาทั้งหมดมีจำนวน: แถว
- การเปรียบเทียบค่าข้อมูล (Data Scaling Verification): จงเลือกดูข้อมูลลูกค้าคนแรก (Row 0) และบันทึกค่าที่เปลี่ยนแปลงไป:
 - ก่อนปรับสเกล (Raw Data): Recency =, Monetary =
 - หลังปรับสเกล (Scaled Data): Recency =, Monetary =

ส่วนที่ 2: ผลการหาจำนวนกลุ่ม

- จำนวนข้อมูล (Rows):
 - K = 1, Inertia =
 - K = 2, Inertia =
 - K = 3, Inertia =
 - K = 4, Inertia =
 - K = 5, Inertia =
- การตัดสินใจ:
 - จากข้อมูลควรเลือกแบ่งลูกค้าออกเป็น กลุ่ม

ส่วนที่ 3: ผลการสร้างโมเดลและตั้งชื่อกลุ่ม

- ค่าเฉลี่ยของแต่ละกลุ่ม (Centroid Analysis):

กลุ่มที่ (Cluster ID)	Recency (Mean)	Frequency (Mean)	Monetary (Mean)
กลุ่ม 0			
กลุ่ม 1			
กลุ่ม 2			

2. เงื่อนไขการตั้งชื่อกลุ่ม (Naming Logic): จงระบุเงื่อนไข IF-ELSE ใน Code ที่ใช้กำหนดชื่อกลุ่ม
- กลุ่ม VIP: ต้องมีค่า
 - กลุ่ม Churned: ต้องมีค่า
 - กลุ่ม General: คือลูกค้ากลุ่มที่เหลือ

ส่วนที่ 4: ผลการพยากรณ์และกลยุทธ์

1. ทดสอบลูกค้า A (1, 25, 15000):
- กลุ่มที่พยากรณ์ได้:
 - กลยุทธ์ที่ระบบเลือก:
2. ทดสอบลูกค้า B (365, 1, 100):
- กลุ่มที่พยากรณ์ได้:
 - กลยุทธ์ที่ระบบเลือก:

ส่วนที่ 5: คำถามท้ายการทดลอง

1. ในไฟล์ `prepare_data.py` ทำไมเราต้องบันทึกไฟล์ `customer_scaler.pkl` แยกออกมา? หากลืมนบันทึกไฟล์นี้ จะส่งผลอย่างไรเมื่อนำไปใช้จริงในไฟล์ `run_model.py`

.....

.....

.....